

EX. NO:3**LINEAR AND CIRCULAR QUEUES****DATE:****QUESTION 3(A):****Implement Enqueue and Dequeue operations on Queue.****AIM:**

To write a C program to implement Enqueue and Dequeue operations on a Linear Queue using an array.

ALGORITHM:**Enqueue Operation**

1. Start.
2. Check whether $\text{rear} == \text{MAX} - 1$.
3. If true, display Queue Overflow.
4. Otherwise, get the value from the user.
5. If $\text{front} == -1$, set $\text{front} = 0$.
6. Increment rear by 1.
7. Insert the value at $\text{queue}[\text{rear}]$.
8. Display the inserted value.
9. Stop.

Dequeue Operation

1. Start.
2. Check whether $\text{front} == -1$ or $\text{front} > \text{rear}$.
3. If true, display Queue Underflow.
4. Otherwise, store $\text{queue}[\text{front}]$ in value.
5. Increment front by 1.
6. If $\text{front} > \text{rear}$, set $\text{front} = \text{rear} = -1$.
7. Display the deleted value.
8. Stop.

PROGRAM:

```
#include <stdio.h>
#define MAX 5

int queue[MAX];
int front = -1;
int rear = -1;

/* Enqueue operation */
void enqueue()
```

```

{
    int value;

    if (rear == MAX - 1)
    {
        printf("Queue Overflow\n");
    }
    else
    {
        printf("Enter the value: ");
        scanf("%d", &value);

        if (front == -1)
        {
            front = 0;
        }

        rear++;
        queue[rear] = value;

        printf("%d inserted into the queue\n", value);
    }
}

/* Dequeue operation */
void dequeue()
{
    int value;

    if (front == -1 || front > rear)
    {
        printf("Queue Underflow\n");
    }
    else
    {
        value = queue[front];
        front++;

        if (front > rear)
        {
            front = rear = -1;
        }

        printf("%d deleted from the queue\n", value);
    }
}

/* Display operation */
void display()
{
    int i;

    if (front == -1)
    {

```

```
        printf("Queue is empty\n");
    }
    else
    {
        printf("Queue elements are: ");

        for (i = front; i <= rear; i++)
        {
            printf("%d ", queue[i]);
        }

        printf("\n");
    }
}

int main()
{
    int choice;

    while (1)
    {
        printf("\n1. Enqueue");
        printf("\n2. Dequeue");
        printf("\n3. Display");
        printf("\n4. Exit");

        printf("\nEnter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
                enqueue();
                break;

            case 2:
                dequeue();
                break;

            case 3:
                display();
                break;

            case 4:
                return 0;

            default:
                printf("Invalid choice\n");
        }
    }

    return 0;
}
```

OUTPUT:

```
[ita2536@mritlin harshan]$ vi ex3.1.c
[ita2536@mritlin harshan]$ cc ex3.1.c
[ita2536@mritlin harshan]$ ./a.out
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter the value: 2
2 inserted into the queue

1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter the value: 3
3 inserted into the queue

1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter the value: 4
4 inserted into the queue

1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 2
2 deleted from the queue

1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 3
Queue elements are: 3 4

1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 4
[ita2536@mritlin harshan]$
```

RESULT:

Thus, the implementation of Enqueue and Dequeue operations on a Linear Queue using an array was executed successfully.

QUESTION 3(B):**Implement Enqueue and Dequeue operations on Circular Queue.****AIM:**

To implement Enqueue and Dequeue operations on a Circular Queue using an array.

ALGORITHM:

Enqueue Operation:

1. Start.
2. Check whether $(\text{rear} + 1) \% \text{MAX} == \text{front}$.
3. If true, display Circular Queue Overflow.
4. Otherwise, get the value from the user.
5. If the queue is empty, set $\text{front} = \text{rear} = 0$.
6. Otherwise, update $\text{rear} = (\text{rear} + 1) \% \text{MAX}$.
7. Insert the value at $\text{queue}[\text{rear}]$.
8. Display the inserted value.
9. Stop.

Dequeue Operation:

1. Start.
2. Check whether $\text{front} == -1$.
3. If true, display Circular Queue Underflow.
4. Otherwise, store $\text{queue}[\text{front}]$ in value.
5. If $\text{front} == \text{rear}$, set $\text{front} = \text{rear} = -1$.
6. Otherwise, update $\text{front} = (\text{front} + 1) \% \text{MAX}$.
7. Display the deleted value.
8. Stop.

PROGRAM:

```
#include <stdio.h>
#define MAX 5

int queue[MAX];
int front = -1;
int rear = -1;

/* Enqueue operation */
void enqueue()
{
    int value;
```

```

if ((rear + 1) % MAX == front)
{
    printf("Circular Queue Overflow\n");
}
else
{
    printf("Enter the value: ");
    scanf("%d", &value);

    if (front == -1)
    {
        front = 0;
        rear = 0;
    }
    else
    {
        rear = (rear + 1) % MAX;
    }

    queue[rear] = value;

    printf("%d inserted into the circular queue\n", value);
}
}

/* Dequeue operation */
void dequeue()
{
    int value;

    if (front == -1)
    {
        printf("Circular Queue Underflow\n");
    }
    else
    {
        value = queue[front];

        if (front == rear)
        {
            front = -1;
            rear = -1;
        }
        else
        {
            front = (front + 1) % MAX;
        }

        printf("%d deleted from the circular queue\n", value);
    }
}

/* Display operation */
void display()

```

```
{
    int i;

    if (front == -1)
    {
        printf("Circular Queue is empty\n");
    }
    else
    {
        printf("Circular Queue elements are: ");

        i = front;

        while (1)
        {
            printf("%d ", queue[i]);

            if (i == rear)
            {
                break;
            }

            i = (i + 1) % MAX;
        }

        printf("\n");
    }
}

int main()
{
    int choice;

    while (1)
    {
        printf("\n1. Enqueue");
        printf("\n2. Dequeue");
        printf("\n3. Display");
        printf("\n4. Exit");

        printf("\nEnter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
                enqueue();
                break;

            case 2:
                dequeue();
                break;

            case 3:
```

```

        display();
        break;

    case 4:
        return 0;

    default:
        printf("Invalid choice\n");
    }
}

return 0;
}

```

OUTPUT:

```

[ita2536@mritlin harshan]$ vi ex3.2.c
[ita2536@mritlin harshan]$ cc ex3.2.c
[ita2536@mritlin harshan]$ ./a.out
--- Queue Menu ---
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter enqueue element: 2

--- Queue Menu ---
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter enqueue element: 4

--- Queue Menu ---
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter enqueue element: 5

--- Queue Menu ---
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 2

--- Queue Menu ---
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 3
Queue is: 4 5

--- Queue Menu ---
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 4
Exit
[ita2536@mritlin harshan]$

```

RESULT:

Thus, the implementation of Enqueue and Dequeue operations on a Circular Queue using an array was executed successfully.